UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Smart Pointer Box<T>

Box é um dos tipos de ponteiros inteligentes de Rust que permite a alocação de espaços no *heap*.

```
fn main() {
    let mut a: Box<i32> = Box::new(5);
    let mut b = Box::new(10);
    *a = 20;
    troca (a, b);
    println!("{a}-{b}"); // Erro: não possui a propriedade
}

fn troca(mut a:Box<i32>, mut b:Box<i32>) {
    let aux = *a;
    *a = *b;
    *b = aux;
    println!("troca:{a} {b}");
}
```

A propriedade (*ownership*) de a e b são transferidas (***move***)

UDESC

Smart Pointer Box<T>

```
fn main() {
    let mut a: Box<i32> = Box::new(5);
    let mut b = Box::new(10);
    *a = 20;
    troca (&mut a, &mut b);
    println!("{a}-{b}");
}

fn troca(a:&mut Box<i32>, b:&mut Box<i32>) {
    let aux = **a;
    **a = **b;
    **b = aux;
}
```

Empréstimo (*borrowing*) de a e b



2 Alternativas para Listas Encadeadas

```
enum No{
    Cons {cab:i32, cauda:Box<NoLista>},
    Nil
}
struct Lista {
    cabeca:Box<NoLista>
}
```

```
struct No {
    elem: i32,
    prox: Option<Box<No>>
}
struct Lista {
    cab: Option<Box<No>>
}
```

UDESC

Métodos

Métodos são um caso especial de funções associadas que recebem como primeiro parâmetro o dado para o qual foi chamado, por meio de self. Existem basicamente 3 formas que pode ser declarado:

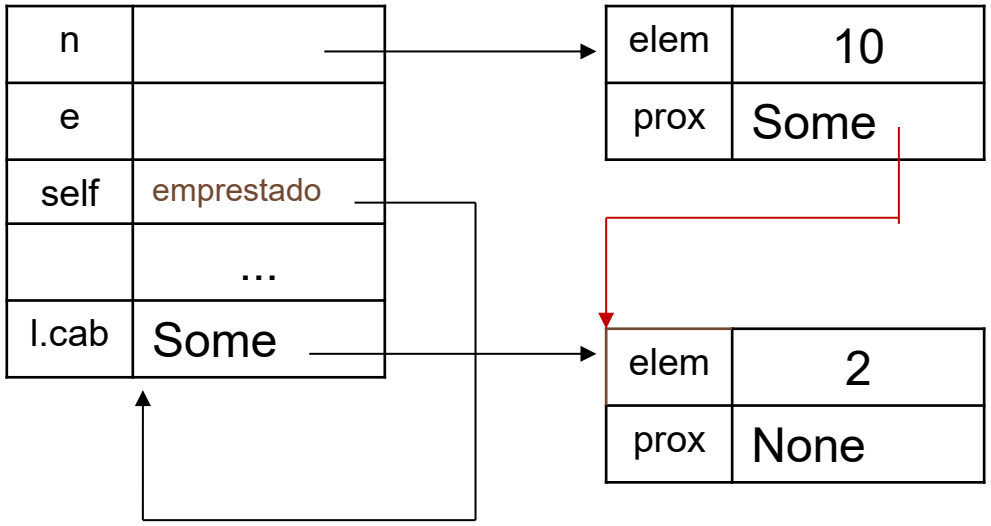
- self – Valor, quando um dado é passado por valor, esse valor é movido para a nova área e a anterior perde a propriedade do dado (exceto em caso de cópia).
- &self - Referência compartilhada (imutável), compartilha a referência para área onde o dado está armazenado, pode acessar o dado mas não possui a sua propriedade.
- &mut self – Referência mutável, recebe uma referência para área onde o dado está armazenado e empresta a propriedade do dado de forma exclusiva. Nenhuma outra parte do código pode acessar essa área e seu valor pode ser modificado. **A única coisa que você não pode fazer com uma referência mutável é movê-la sem substituí-la.**

Listas Encadeadas

```
#[derive (Debug)]
struct No {
    elem: i32,
    prox: Option<Box<No>>
}
#[derive (Debug)]
struct Lista {
    cab: Option<Box<No>>
}

impl Lista {
    fn new() -> Self {
        Lista {cab :None}
    }

    fn ins_cab_erro(&mut self, e:i32) {
        let n = Box::new(No {elem:e, prox: self.cab});
        self.cab = Some(n);                    //^^^^^^^^ Erro: Não é possível
    }                                          //               mover um conteúdo
}                                              //               emprestado
```



Listas Encadeadas

```
#[derive (Debug)]
struct No {
    elem: i32,
    prox: Option<Box<No>>
}
#[derive (Debug)]
struct Lista {
    cab: Option<Box<No>>
}

impl Lista {
    fn new() -> Self {
        Lista {cab :None}
    }
    fn ins_cab(&mut self, e:i32) {
        let n = Box::new(No {elem:e,
                             prox: std::mem::replace(&mut self.cab, None)});
        self.cab = Some(n);
    }
}
```

replace(dest:&mut T, src:T) -> T Troca o conteúdo da área referenciada por *dest* pelo conteúdo de *src*, retorna o conteúdo armazenado anteriormente na área referenciada por *dest*.



Listas Encadeadas

```
#[derive (Debug)]
struct No {
    elem: i32,
    prox: Option<Box<No>>
}
#[derive (Debug)]
struct Lista {
    cab: Option<Box<No>>
}

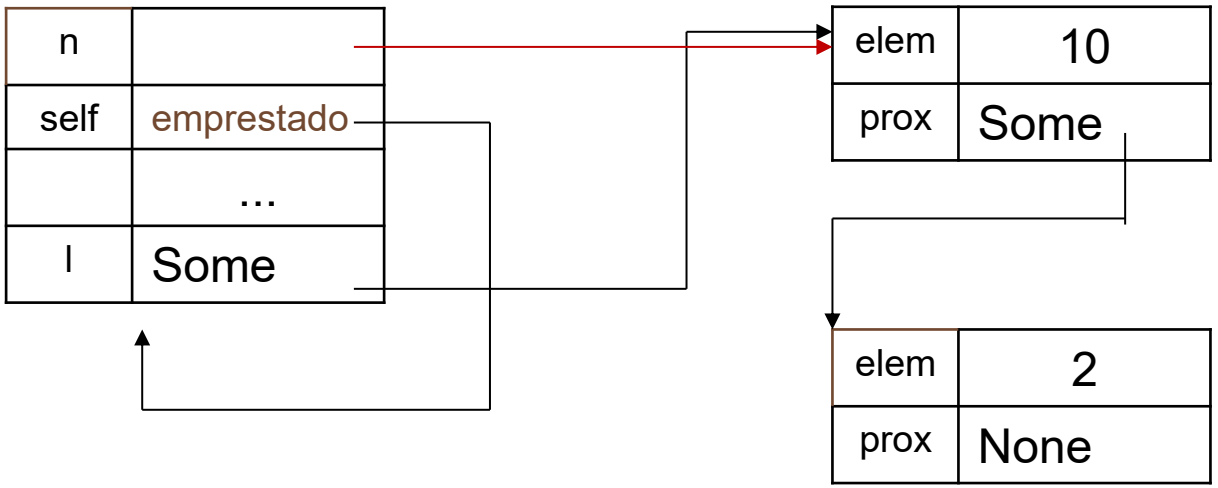
impl Lista {
    fn new() -> Self {
        Lista {cab :None}
    }
    fn ins_cab(&mut self, e:i32) {
        let n = Box::new(No {elem:e, prox: self.cab.take()});
        self.cab = Some(n);
    }
}
```

take(&mut self) -> Option<T> Troca o conteúdo da área referenciada por *self* por None, retorna o conteúdo armazenado na área referenciada por *self*.



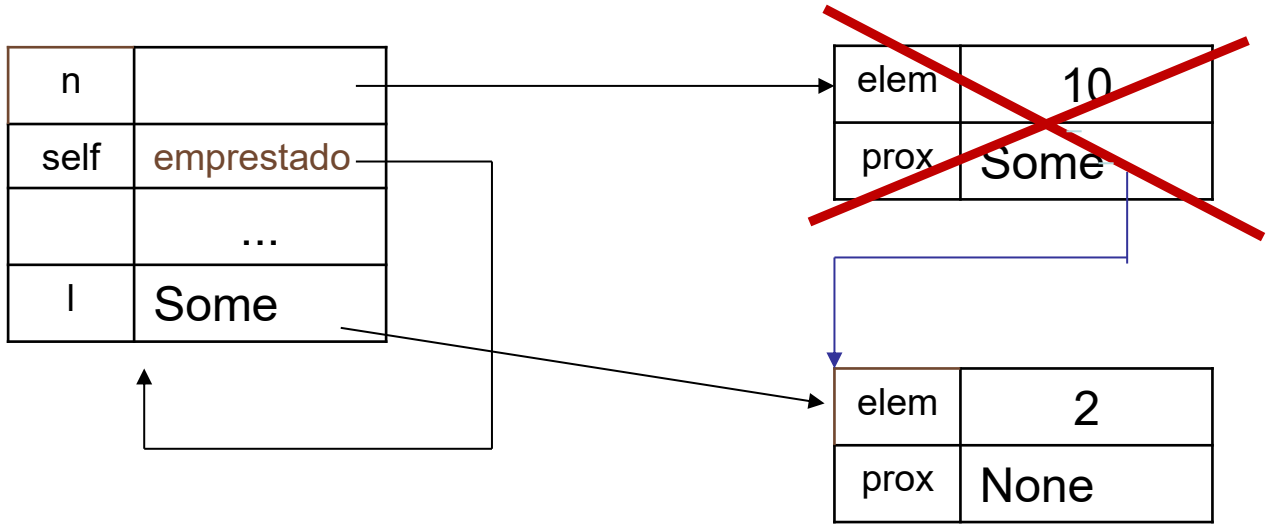
Listas Encadeadas

```
fn remover_cabeca_erro (&mut self) -> Option<i32> {
        match self.cab {
            None => None,
            Some(n:Box<No>) => {  //Erro
                self.cab = n.prox;
                Some(n.elem)
            }
        }
}
```



Listas Encadeadas

```
fn remover_cabeca_erro (&mut self) -> Option<i32> {
        match self.cab.take() {
            None => None,
            Some(n:Box<No>) => {
                self.cab = n.prox; // A propriedade é transferida
                Some(n.elem)       // n.prox não tem mais acesso
            }                      // ao No
        }
}
```

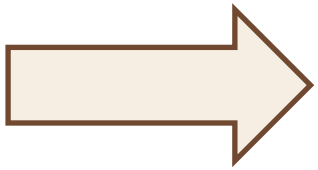


UDESC

Tipos Genéricos

Permitem definir funções e estruturas polimórficas com tipos parametrizados que são resolvidos em tempo de compilação:

```
#[derive (Debug)]
struct No {
    elem: i32,
    prox: Option<Box<No>>
}
#[derive (Debug)]
struct Lista {
    cab: Option<Box<No>>
}
...
let mut l1 = Lista::new();
l1.ins_cab(20);
```



```
#[derive (Debug)]
struct No<T> {
    elem: T,
    prox: Option<Box<No<T>>>
}
#[derive (Debug)]
struct Lista<T> {
    cab: Option<Box<No<T>>>
}
...
let mut l1:Lista<i32> = Lista::new();
l1.ins_cab(20);
```

UDESC

Listas com Tipos Genéricos

```
#[derive (Debug)]
struct No<T> {
    elem: T,
    prox: Option<Box<No<T>>>
}
#[derive (Debug)]
struct Lista<T> {
    cab: Option<Box<No<T>>>
}
impl<T> Lista<T> {
    fn new() -> Self {
        Lista {cab :None}
    }
    fn ins_cab(&mut self, e:T) {
        let n = Box::new(No {elem:e, prox: std::mem::replace(&mut self.cab, None)});
        self.cab = Some(n);
    }
}
```



Listas usando União Disjunta

```
#[derive (Debug)]
enum No<T>{
    Cons {cab:T, cauda:Box<NoLista<T>>},
    Nil
}
#[derive (Debug)]
struct Lista<T> {
    cab:Box<No<T>>
}
use std::mem::replace;
use No::{*};
impl<T> Lista<T> {
   fn new() -> Self {
        Lista {cab : Box::new(Nil)}
    }
   fn ins_cab(&mut self, e:T) {
        let n = Box::new(Cons {cab:e, cauda: replace(&mut self.cabeca, Box::new(Nil))});
        self.cab = n;
   }
}
```

Listas usando União Disjunta

```
impl<T:Ord> Lista<T> {
   fn ins_ordenado (&mut self, e:T) {
        let p = replace(&mut self.cabeca, Box::new(Nil));
        self.cab = Self::ins_ord(p, e);
    }

   fn ins_ord (p:Box<NoLista<T>>, e:T) -> Box<NoLista<T>>{
        match *p {
            Nil => Box::new(Cons {cab:e, cauda:p}),
            Cons {cab:x, cauda:xs} => if e < x {Box::new(Cons {cab:e,
                                                  cauda:Box::new(Cons {cab:x,cauda:xs})})}
                                      else {Box::new (Cons {cab:x,
                                            cauda:Self::ins_ord (xs, e)})}
        }
    }
}
```

UDESC

Listas usando União Disjunta

```
#[derive (Debug, PartialEq, Eq, PartialOrd, Ord)]
enum Semana {Dom, Seg, Ter, Qua, Qui, Sex, Sab}

fn main() {
    let mut l = Lista::new();

    l.ins_ordenado(Qua);
    l.ins_ordenado(Seg);
    l.ins_ordenado(Dom);
    println!("{l:?}");
}
```